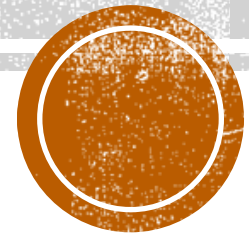




Module: Développer en Front-end

Le langage JSX-Composant React



Filière: Développement digital – option web full stack

Réalisé par: ELMAUDNI

Définition

- JSX, qui signifie **JavaScript XML**, est une **extension de JavaScript** couramment utilisée dans le développement web pour créer des interfaces utilisateur. C'est une syntaxe qui **ressemble à du HTML**, mais qui est en réalité interprétée par JavaScript.
- JSX est une **forme d'écriture des éléments React**, plus **simple** à lire et à écrire que les instructions `React.createElement()`
- les deux propriétés de base pour ce qui est des balises utilisées :
 - Toute balise commençant par une **minuscule** (div, span, label, etc.) est réservé aux **éléments HTML**. Ces éléments sont déclarés par React DOM, et vous obtiendrez une erreur si vous utilisez un élément inexistant.
 - Toute balise commençant par une **majuscule** (Greetings, App, etc.) doit être déclarée explicitement, ce doit donc être un élément du scope courant : fonction déjà déclarée, composant importé d'une bibliothèque ou d'un autre fichier...

Exemple:

- JSX est donc une forme nouvelle d'écriture des éléments React. Par exemple, voici un paragraphe contenant Hello React dans son texte.
- **var p = <p>Hello React</p>; // Code JSX**

Il sera traduit par **Babel** en code JavaScript standard utilisant les fonctions de création d'éléments React.

Voici comment il sera généralement transformé .

- **var p = React.createElement("p", null, "Hello React");**

JSX est donc une forme nouvelle d'écriture des éléments React .
L'utilisation de JSX permet de créer et de rendre ce composant de manière très lisible et expressive.

- Pour afficher ce paragraphe, on utilise l'outil Babel pour interpréter le code JSX et de le transformer en interne en code JavaScript compréhensible par le navigateur.

```

<html>
  <head>
    <script
src="https://unpkg.com/react@16/umd/react.development.js"
crossorigin></script>
    <script src="https://unpkg.com/react-dom@16/umd/react-
dom.development.js" crossorigin></script>
    <script
src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
  </head>
  <body>
    <div id="app"></div>
  </body>

  <script type="text/babel">
    var p = <p>Hello React</p>; // Code JSX
    console.log(p); // Affichage dans la console de l'élément
    React
    ReactDOM.render(p, document.getElementById("app"));
  </script>
</html>

```

- **NB:** l'outil Babel pour interpréter le code JSX ralentit le programme. Par conséquent, l'utilisation de Babel ne peut être viable que dans le cadre de l'écriture du programme (en mode développement). Cet outil ne peut pas être utilisé dans le cadre d'un déploiement (mode production). Dans ce dernier cas, on utilisera d'autres outils tels que Webpack pour créer un package plus compact.

Exemples: Elements JSX

- Créer une liste de cinq éléments en JSX

```
const root = ReactDOM.createRoot(document.getElementById('root'));  
var liste = <ul>  
  <li> Element1 </li>  
  <li> Element2 </li>  
  <li> Element3 </li>  
  <li> Element4 </li>  
  <li> Element5 </li>  
</ul>;  
root.render(liste);
```

- Le code JSX est facile à lire et à écrire. Il s'écrit comme du code HTML, mais il est saisi dans la partie réservée au code JavaScript (dans la balise <script>).

Utilisation d'instructions JavaScript dans le code JSX

- On peut utiliser des instructions JavaScript dans du code JSX, à condition d'entourer les instructions JavaScript avec **des accolades**. Chaque instruction entourée d'accolades est évaluée par le navigateur, et son résultat est inséré en lieu et place de l'instruction JavaScript évaluée. Ceci permet de créer du code JSX qui s'adapte aux conditions définies dans le programme.
- Exemple:

```
Let message = "Bonjour!";  
var texte= <div>  
    <h1>Bienvenue dans notre application</h1>  
    <p>{message}</p>  
    </div>;  
root.render(texte);
```

Ajouts d'attributs dans le code JSX

- Le code JSX comporte les éléments qui seront affichés dans la page HTML. Ces éléments peuvent avoir des attributs tels que id, style ou className
- l'attribut **class** est remplacé par l'attribut **className**.

```
<style type="text/css">
.red {
color : red;
}
</style>
```

```
var liste = <ul id="list1"
className="red">
<li> Element1 </li>
<li> Element2 </li>
<li> Element3 </li>
<li> Element4 </li>
<li> Element5 </li>
</ul>;
root.render(liste);
```

Ajout de l'attribut style en JSX

- La syntaxe à utiliser pour insérer l'attribut style est légèrement différente
- le style doit dans ce cas être défini au moyen d'un **objet JavaScript**.

```
var styleListe = { listStyleType:"none", color:"red" };
var liste = <ul id="list1" style={styleListe}>
  <li> Element1 </li>
  <li> Element2 </li>
  <li> Element3 </li>
  <li> Element4 </li>
  <li> Element5 </li>
</ul>;
root.render(liste);
```

- La propriété list-style-type s'écrit dans le code JavaScript sous la forme listStyleType, en remplaçant comme d'habitude chaque tiret et la lettre qui le suit par une majuscule.
- L'instruction JavaScript {styleListe} indique de calculer la valeur de l'expression styleListe, puis d'affecter cette valeur au style de l'élément dans le code JSX.

Ajout de l'attribut style en JSX

- L'attribut style est défini en utilisant les caractères `{{ et }}`: Les accolades extérieures indiquent que l'expression à l'intérieur est une expression JavaScript. De plus, le style est un objet JavaScript.

```
var liste = <ul id="list1" style={{listStyleType:"none",  
color:"red"}}>  
  <li> Element1 </li>  
  <li> Element2 </li>  
  <li> Element3 </li>  
  <li> Element4 </li>  
  <li> Element5 </li>  
</ul>;  
root.render(liste);
```

Exemple : Insérer les éléments de liste définis dans un tableau

- On peut améliorer notre code en insérant les éléments de liste au moyen d'un bloc de code JavaScript. Les éléments de la liste sont placés dans un tableau `elems` qui est ensuite parcouru par le code JavaScript et JSX.
- toute expression JavaScript doit être encadrée par des accolades

```
var color = "red";
var styleListe = { listStyleType:"none", color:color };
var elems = ["Element1", "Element2", "Element3", "Element4",
"Element5"];
var liste = <ul id="list1" style={styleListe}>
{
elems.map(function(elem,index) {
return <li key={index}>{elem}</li>
})
}
</ul>;
root.render(liste);
```

- Ajouter un attribut `key` lors de l'utilisation de `map` en JSX est essentiel pour permettre à React d'identifier de manière unique chaque élément généré, ce qui facilite la mise à jour et l'optimisation des performances lors de la réconciliation de l'interface utilisateur. L'attribut `key` permet d'éviter un avertissement lors de l'exécution du code (message d'erreur «Each child in an array or iterator should have a unique "key" prop.»).

Utiliser la notation ES6 pour définir la fonction

```
var color = "red";
var styleListe = { listStyleType:"none", color:color };
var elems = ["Element1", "Element2", "Element3", "Element4",
"Element5"];
var liste = <ul id="list1" style={styleListe}>
{
elems.map((elem, index) => {
return <li key={index}>{elem}</li>
}))
}
</ul>;
root.render(liste);
```

```
elems.map((elem, index) =>
<li key={index}>{elem}</li>
)
```

Composant en React

- Un composant est fonction qui retourne de JSX. On va donc ici apprendre à créer le composant **<ListeElements>** qui représentera la liste **** contenant les éléments ****.
- **Créer une fonction qui retourne du code JSX** : Un élément défini en JSX, tel que **<ListeElements/>**, doit obligatoirement commencer par une majuscule

```
var elems = ["Element1", "Element2", "Element3", "Element4",  
"Element5"];  
function ListeElements() {  
  return <ul>  
  {  
    elems.map(function(elem, index) {  
      return <li key={index}>{elem}</li>;  
    })  
  }  
</ul>  
}  
root.render(<ListeElements/>);
```

Autrement : Retourner uniquement les éléments dans la fonction (sans l'élément)

- La fonction ne retourne plus l'élément , donc les accolades qui servaient à indiquer le code JavaScript à l'intérieur du code JSX ne sont ici plus nécessaires

```
var elems = ["Element1", "Element2", "Element3", "Element4", "Element5"];
var ListeElements = function() {
  return elems.map(function(elem, index) {
    return <li key={index}>{elem}</li>;
  })
}
root.render(<ul><ListeElements/></ul>);
```

- NB: Même si l'affichage de la liste est identique au précédent, on voit ici que les éléments React et <ListeElements> ont été inversés dans l'arborescence.

Transmettre des attributs dans un composant React

- On peut transmettre des attributs aux composants React définis ici en JSX. Par exemple, le tableau **elems** pourrait être transmis dans l'attribut **elems** de l'élément JSX. On peut créer les attributs que l'on souhaite dans un élément JSX, ces attributs seront transmis en paramètres de la fonction de traitement dans l'objet **props**.

```
var elems = ["Element1", "Element2", "Element3", "Element4", "Element5"];
var ListeElements = function(props) {
  return <ul>
  {
    props.elems.map(function(elem, index) {
      return <li key={index}>{elem}</li>;
    })
  }
</ul>
}
root.render(<ListeElements elems={elems}/>);
```

- L'attribut **elems** est défini lors de l'écriture de l'élément JSX **<ListeElements elems={elems}/>**. Les attributs d'un élément défini par une fonction sont transmis dans l'objet **props** en paramètres de la fonction. Ainsi, pour accéder à l'attribut **elems** dans la fonction, on utilise **props.elems**.

Transmission de l'attribut style dans un composant

- Transmettons maintenant l'attribut style dans le composant. Le style indiqué sera affecté aux éléments de la liste.

```
var elems = ["Element1", "Element2", "Element3", "Element4", "Element5"];
var ListeElements = function(props) {
  return <ul>
  {
    props.elems.map(function(elem, index) {
      return <li key={index} style={props.style}>{elem}</li>;
    })
  }
  </ul>
}
root.render(<ListeElements elems={elems} style={{color:"red"}}/>);
```

- Le style est indiqué comme d'habitude sous forme d'objet JSON (ici, { **color:"red"** }), et comme c'est une instruction JavaScript, il faut l'entourer des accolades, d'où les doubles accolades que l'on peut voir ici dans l'élément JSX.
- Ce style est récupéré dans la fonction au moyen du paramètre props, et il est accédé à l'aide de props.style dans l'élément JSX définissant chaque élément .

Écriture du programme en déstructurant l'objet props (en ES6)

```
var elems = ["Element1", "Element2", "Element3", "Element4", "Element5"];
var ListeElements = function({elems,style}) {
  return <ul>
  {
    elems.map(function(elem, index) {
      return <li key={index} style={style}>{elem}</li>;
    })
  }
  </ul>
}
root.render(<ListeElements elems={elems} style={{color:"red"}}/>);
```

- On accède maintenant directement aux variables **elems** et **color** précédemment définies comme propriétés dans l'objet **props**.

Écrivons le composant `<Element>` utilisé par le composant `<ListeElements>`.

```
var elems = ["Element1", "Element2", "Element3", "Element4",
"Element5"];
function Element(props) {
return <li style={{color:props.color}}>{props.elem}</li>;
}
function ListeElements (props) {
return <ul>
{
props.elems.map(function(elem, index) {
return <Element key={index} elem={elem} color={color} />
})
}
</ul>
}
root.render(<ListeElements elems={elems} color="red" />);
```

- Le composant `<Element>` est lui aussi créé avec une fonction dans laquelle les attributs `index`, `color` et `elem` sont transmis en paramètres dans l'objet `props` (ici, utilisé sous forme destructurée). L'attribut `key` est utilisé pour éviter l'erreur classique de React indiquant que cet attribut est obligatoire. Toutefois, il ne sert qu'à mettre une clé différente sur les éléments issus d'une fonction d'itération, donc il est utilisé dans l'écriture de l'élément `<Element>` (écrit dans une boucle d'itération), mais pas dans les paramètres de la fonction `Element()`.

Créer un composant avec une classe

- On peut créer des éléments React (et JSX) à partir d'une classe dérivant de la classe **React.Component**.
- Créons maintenant deux classes correspondant aux deux composants utilisés précédemment (<Element> et <ListeElements>). Ces deux classes dérivent de la classe **React.Component**.
- Créer les classes associées aux composants <Element> et <ListeElements>

```
var elems = ["Element1", "Element2", "Element3", "Element4",  
"Element5"];  
class Element extends React.Component {  
  constructor(props) {  
    super(props);  
  }  
  render() {  
    return <li  
      style={{color:this.props.color}}>{this.props.elem}</li>  
    }  
  }  
}
```

```
class ListeElements extends React.Component {  
  constructor(props) {  
    super(props);  
  }  
  render() {  
    return <ul>  
    {  
      this.props.elems.map((elem, index) => {  
        return <Element key={index} elem={elem} color={this.props.color} />  
      })  
    }  
    </ul>  
  }  
}  
root.render(<ListeElements elems={elems} color="red" />);
```

- L'instruction **root.render()** est la même que celle utilisée dans la section précédente. On transmet dans la classe **ListeElements** les attributs **elems** et **color**, utilisés dans la classe via l'objet **this.props** qui les contient.

Règles d'écriture du code JSX

- **Un seul élément parent peut être retourné :**
- Plusieurs éléments JSX de même niveau ne peuvent pas être retournés simultanément, il est obligatoire qu'ils soient encapsulés dans un élément parent, qui sera celui retourné (pour être unique), les autres éléments étant ses enfants. En général on utilise un élément `<div>` englobant l'ensemble, mais React propose aussi d'utiliser un composant `<React.Fragment>` jouant ce rôle.
- Utilisons le composant `<React.Fragment>` pour englober un ensemble de trois éléments `<div>` sans parents. L'élément `<React.Fragment>` va devenir le parent des trois éléments `<div>`, sans apparaître pour autant dans l'arborescence des éléments React.

```
function ListeElements(props) {  
  return <React.Fragment>  
    <div>Element1</div>  
    <div>Element2</div>  
    <div>Element3</div>  
  </React.Fragment>  
}  
root.render(<ListeElements />);
```

- **<> (version abrégée) :**
- La version abrégée `<>` ne vous permet pas de spécifier des clés ou des propriétés. C'est une syntaxe plus concise et est généralement préférée lorsque vous n'avez pas besoin de ces fonctionnalités.